Tetris 2.0: Software Design

This document explains how the game is designed inside: its goals, its parts, the main data structures and algorithms, and the reasons behind the main choices. For the big-picture view (components, runtime flow, deployment), see `ARCHITECTURE.md`. For how to play, see `Tetris_User_Manual.md`.

1. Goals and Scope

What the software is: a single-player Tetris clone for the desktop, written in Python with pygame, with local player accounts, per-player high scores and settings, and optional high score sync through a small self-hosted server.

Design goals

Goal	What it means in the code
Playable and correct	Follows the standard Tetris rules: 7-bag, SRS rotation with wall kicks, lock delay, T-spins, combos, back-to-back
Testable	Game rules, accounts, scoring, and sync live in plain classes and functions that run without a window
Safe by default	Passwords are hashed, files are validated on load, the network is optional and never blocks play for long

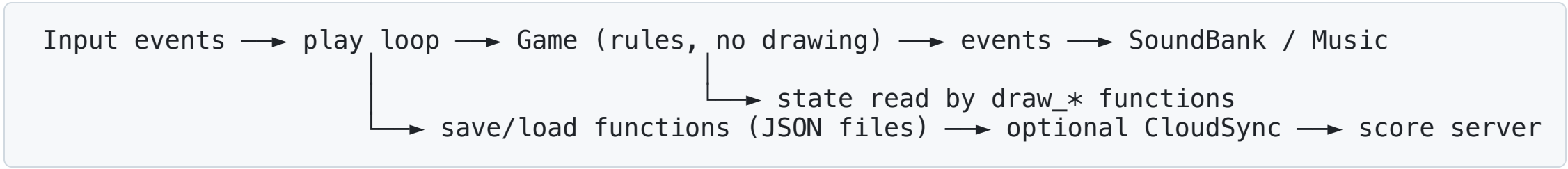
2. Requirements Summary

Functional

1. Players create accounts, sign in, and sign out with the mouse and keyboard.
2. The game deals pieces from a 7-bag and shows a hold box and the next 3 pieces.
3. Pieces move, rotate (SRS with wall kicks), soft drop, hard drop, and lock after a delay.
4. Completed rows clear, score points, and raise the level every 10 lines.
5. T-spins, combos, and back-to-back clears earn bonus points.
6. A Tetris or T-spin line clear earns a row bomb (max 3); pressing B blasts the bottom 2 rows.
7. The game plays sound effects and looping music that speeds up with the level, and can be muted.
8. Players can pause, restart, quit, and sign out. Three themes are available.

3. Design Overview

The code follows one rule: **keep game logic separate from drawing and input.**



- The **Game** object holds all the rules and state. It has no idea how it is drawn.
- **draw_*** functions only read the state and paint it.
- The **play loop** turns key and mouse events into Game actions, advances time, plays sounds, and draws.
- Game rules talk to sound through a simple **event list** (`game.events`), not by calling the audio code.

Because of this split, almost all rules can be tested by calling methods directly, with no window.

4. Module Breakdown

4.1 `tetris_ver2.py` (the game)

The file is organized in labeled sections, top to bottom:

Section	Contents
Constants	Board size, window layout, colors, file paths, limits
Themes	`THEMES`, `apply_theme`, `next_theme`
Piece data	`PIECE_SHAPES`, rotation states, SRS wall kick tables
Rules constants	Scoring tables, lock delay, bomb limits, `gravity_interval`
Core classes	`Piece`, `Board`, `SevenBag`, `Game`
Sound	`SOUND_NOTES`, `synth`, `SoundBank`
Music	`music_speed`, `music_notes`, `Music`

5. Class Design

5.1 `Piece`

One falling tetromino: `kind` (I, O, T, S, Z, J, L), `rotation` (0 to 3), and its `row` and `col`. `cells()` returns the absolute board cells it covers. A piece has no knowledge of the board; the board and game decide if a position is allowed.

5.2 `Board`

Owns the 10 x 20 grid of locked cells (each cell is `None` or the kind letter that was locked there).

Method	Responsibility
`is_valid(cells)`	True if every cell is inside the walls and floor and not already filled. Rows above the top are allowed so pieces can spawn partly off the top

6. Key Algorithms and Rules

6.1 Piece rotation (SRS with wall kicks)

Each piece's four rotation states are built once from the base shape by rotating the cell grid clockwise. When a rotation is blocked, the game tries the standard SRS offset list for that rotation (one table for J, L, S, T, Z and another for I). The first offset that fits is used, and its index is remembered as `_last_kick` (a "kick 4" rotation counts toward a full T-spin).

6.2 7-bag randomizer

See 5.3. It guarantees at most 12 pieces between two of the same kind and removes long droughts.

6.3 Gravity

```
seconds_per_row = max(0.05, (0.8 - (level - 1) * 0.007) ** (level - 1))
```

7. Data Design

All data is JSON, stored next to the game.

File	Shape	Written by
`users.json`	`{ "username_lowercase": { "name": "...", "salt": "hex", "hash": "hex", "iterations": 600000 } }`	`register_user`
`scores/<user>.json`	`[900, 500, 300, 200, 100]` (top 5, high to low)	`save_score`, `write_scores`
`settings/<user>.json`	`{ "theme": "light", "muted": false }`	`save_settings`
`cloud_scores.json`	`{ "username_lowercase":`	`ScoreStore`

8. Screens and Control Flow

```
main()
 └─ loop: apply light theme → run_login() → user or None
        └─ play(user) → "logout" (back to login) or "quit" (exit)
```

- `run_login` runs the `LoginScreen` until someone signs in or quits.
- `play` wraps `_play` and always stops the music when it ends.
- `_play` is the frame loop: read events, apply them to `Game`, call `game.update(dt)`, play queued sound events, update music, draw. At game over it saves the score and syncs once.

Frame time `dt` comes from `clock.tick(FPS) / 1000`. Holding the Down key sets a flag on key down and clears it on key up, so soft drop repeats each frame.

9. Error Handling

Situation	Behavior
No audio device	`SoundBank` and `Music` become silent; the game runs normally
Missing, empty, or corrupt data file	Defaults are used; the game keeps running
Hand-edited `users.json`	Records are re-validated; bad entries are ignored
Cloud server unreachable, wrong key, bad reply	`CloudSync` returns `None`; local scores are kept
Server receives bad input	Replies 400, 401, or 404 with a short JSON error; nothing is stored
Window closed with the X button	Exits without saving the unfinished run (Pause > Quit and Sign Out do save)

10. Security Design

Area	Design
Passwords	Never stored. Each account has a random 16-byte salt and a PBKDF2-HMAC-SHA256 hash (600,000 iterations); the iteration count is stored per account so it can be raised later
Sign in errors	One generic message, so it never says whether the username or password was wrong
Usernames	3 to 12 letters, numbers, underscores. Re-checked when files are loaded and before any use in a file path or URL, which blocks path tricks such as `../`
File paths	Built from validated, lowercased usernames only
Network	Off unless `TETRIS_CLOUD_URL` is set. Only `http://` and `https://`

11. Design Decisions and Trade-offs

Decision	Why	Cost
Rules in `Game`, drawing in `draw_*`	Rules can be tested without a window	Drawing code reads many `Game` fields
Event list instead of calling audio from rules	Rules stay free of audio code and testable	One more list to clear each frame
Sound generated in code	No audio files to ship or lose	Simple beeps, not recorded music
JSON files, not a database	Simple, readable, no install	Not safe for many users at once; no queries
Optional cloud sync, offline first	The game always works	Scores are only shared when sync is on

12. Design Principles in Use

- **Separation of concerns:** rules, drawing, input, storage, and network are separate groups of code.
- **Single responsibility:** `Piece` is a position, `Board` is a grid, `SevenBag` is a randomizer, `Game` coordinates them.
- **Fail soft:** every outside source (files, audio, network) has a safe fallback.
- **Validate at the edge:** names, files, and requests are checked when they enter, not later.
- **Data-driven rules:** scoring, kicks, sounds, and themes are tables, so changing them does not change logic.
- **Defaults off for risk:** cloud sync is off unless set up, and the server listens only on the local machine.

13. Testing Strategy

Level	Examples
Unit	Scoring, bag contents, line clearing, kick use, gravity, merge rules, password hashing
Component	`LoginScreen` flows, score and settings files, `Music` state changes, `ScoreStore` persistence
Integration	Real HTTP server on a free port with the real client; scripted play session driving the whole loop with mocked events
Layout	Buttons and panels fit inside the window and do not overlap
Safety	Bad usernames, bad request bodies, wrong keys, oversize bodies, damaged files

Run: `python3 -m unittest test_tetris_render test_tetris_auth`

14. Known Limitations and Future Work

Item	Notes
Single file	`tetris_ver2.py` is about 1,600 lines. Splitting into `rules.py`, `audio.py`, `accounts.py`, `ui.py` would help readability
Global theme state	A `Theme` object passed to the drawing functions would remove the globals
Blocking sync	A background thread would remove the short pause when the server is slow
Closing with X loses a run	Could save on the quit event
No account recovery	By design, since there is no email or server login